

Concepts: FDA behavior-clone system

("parallel")

Every concept used in the build, with the exact code. Project: a five-stage pipeline (Sense → Understand → Clone → Simulate → Surface) that tracks FDA regulatory changes and predicts stakeholder behavior with three behavior-pattern clones (enforcement, reviewer, sponsor), built on parallel.ai's API suite.

Repo: shripathmit/parallel. Live: FastAPI on Railway, auto-deployed from GitHub.

1. LLM-powered web search (Parallel Search) — the Sense layer

Concept. Search APIs that take a natural-language *objective* plus explicit search queries and return grounded results, instead of keyword matching. Used to discover FDA publications (warning letters, guidances, recalls).

```
def search(self, objective, search_queries, max_results=10):  
  
    return self._post("/v1/search", {  
  
        "objective": objective,  
  
        "search_queries": search_queries,  
  
        "max_results": max_results,  
  
    })
```

2. Structured web extraction (Parallel Extract) — Sense

Concept. Fetch a URL and extract exactly the fields you name, returned as structured data.

Turns unstructured FDA pages into machine-readable records (title, date, cited CFR parts, affected product codes).

```
def extract(self, url, objective):

    return self._post("/v1beta/extract", {

        "url": url,

        "objective": objective,

    })
```

3. Agentic task execution with output contracts (Parallel Task) — Understand

Concept. Long-running agentic tasks: give an objective, optionally constrain the output with a JSON Schema, poll until done. The schema is the contract that makes downstream code reliable — the model must return parseable, typed data.

```
def task_run(self, objective, output_schema=None, webhook_url=None):

    body = {"objective": objective}

    if output_schema:

        body["output_schema"] = output_schema

    if webhook_url:

        body["webhook_url"] = webhook_url

    return self._post("/v1/tasks/runs", body)

def wait_for_task(self, run_id, timeout=600, interval=10):
```

```
# poll task_result(run_id) until terminal state or timeout
```

Every analysis is validated against CHANGE_ANALYSIS_SCHEMA (see concept 12).

4. Chat completions with system prompts (Parallel Chat) — Clone runtime

Concept. The clone *is* a system prompt: a role definition plus behavioral grounding instructions. At inference time the analyzed change goes in as context and the model responds in character as that stakeholder.

```
def chat(self, messages, system=None, model="parallel-chat"):

    body = {"model": model, "messages": messages}

    if system:

        body["system"] = system

    return self._post("/v1/chat/completions", body)
```

5. Entity discovery at scale (Parallel FindAll) — corpus building

Concept. "Find me all entities matching these criteria" — bulk discovery across the web in one call. Used to assemble the training corpus of FDA records per stakeholder before pattern mining.

```
def findall(self, objective, criteria=None):

    body = {"objective": objective}

    if criteria:

        body["criteria"] = criteria
```

```
return self._post("/v1beta/findall/runs", body)
```

6. Change monitoring + webhooks (Parallel Monitor) — continuous sensing

Concept. Declarative monitors: "watch this URL/topic, hit my webhook when it changes."
Push instead of poll. The webhook receiver diffs the new capture against the stored baseline and emits a change event.

```
def create_monitor(self, name, objective, url, frequency, webhook_url):

    return self._post("/v1alpha/monitors", {

        "name": name,

        "objective": objective,

        "url": url,

        "frequency": frequency,    # TODO: verify accepted values

        "webhook_url": webhook_url, # TODO: verify signing scheme

    })
```

7. Batched task execution (Parallel Task Groups) — Simulate at scale

Concept. Submit many tasks as one group and collect results together. Used to run large simulation batches (many events × many clones) without N round trips.

```
def task_group(self, tasks):

    return self._post("/v1beta/tasks/groups", {"tasks": tasks})
```

8. Behavior cloning via pattern mining

Concept. Instead of fine-tuning a model per stakeholder, distill *behavior patterns* from historical records with an LLM, then use them as the clone's evidence base. Cheaper than training, inspectable, and refreshable without retraining. The mining pipeline is FindAll (corpus) → Task (distill):

```
def mine_patterns(client, stakeholder, seed_query=None, max_entities=25):

    query = seed_query or SEED_QUERIES[stakeholder]

    found = client.findall(

        objective=f"Find FDA records showing {stakeholder} behavior patterns: {query}",

        criteria={"stakeholder": stakeholder, "topic": query,

            "max_entities": max_entities},

    )

    entities = found.get("entities", found.get("results", [])) or []

    run = client.task_run(

        objective=(f"Distill recurring {stakeholder} behavior patterns from these "

            f"FDA records. Each pattern needs a name, the trigger "

            f"conditions, and evidence citations.\n\nRecords:\n"

            f"{json.dumps(entities)[:15000]}"),

        output_schema={"type": "object", "required": ["patterns"],

            "properties": {"patterns": {"type": "array",

                "items": BEHAVIOR_PATTERN_SCHEMA}}},
```

```
)

output = client.wait_for_task(run.get("run_id", run.get("id")))

return output.get("patterns", []) if isinstance(output, dict) else []
```

Mined patterns persist into the library so future clones reason from them:

```
def mine_and_store(client, data_dir, stakeholder):

    mined = mine_patterns(client, stakeholder)

    store = PatternStore(data_dir)

    for m in mined:

        store.add(Pattern(name=m.get("pattern_name", f"{stakeholder}-pattern"),

                           stakeholder=m.get("stakeholder", stakeholder),

                           description=m.get("description", ""),

                           evidence_citations=m.get("evidence_citations", []),

                           confidence=float(m.get("confidence", 0.5)),

                           support=1, demo=False))
```

9. Deterministic pattern matching — the retrieval algorithm

Concept. Retrieval without embeddings or an LLM call: score every pattern by *trigger coverage* $\times$ *confidence*, partitioned per stakeholder so behaviors never cross-contaminate. Fully explainable — each fired pattern reports its score, matched triggers, and confidence.

```
def match_patterns(analysis, patterns, stakeholder, top_k=3):
```

```

text = " ".join([

    analysis["summary"],

    analysis["change_type"],

    " ".join(analysis["affected_product_codes"]),

    " ".join(analysis["affected_submission_types"]),

]).lower()

scored = []

for p in patterns:

    if p.stakeholder != stakeholder or not p.triggers:

        continue

    matched = [t for t in p.triggers if t.lower() in text]

    if not matched:

        continue

    score = (len(matched) / len(p.triggers)) * p.confidence

    scored.append((p, round(score, 3), matched))

scored.sort(key=lambda s: s[1], reverse=True)

return scored[:top_k]

```

Example pattern from the library:

```

Pattern(

    name="inspection-refusal-escalation",

    stakeholder="enforcement",

```

```
triggers=["501(j)", "refused inspection", "denied inspection",

"delayed inspection", "refusal"],

description="When a firm delays, denies, or refuses FDA inspection, CDER deems "

"the drugs adulterated under 501(j) and escalates faster toward "

"import alert or injunction than for CGMP findings alone.",

evidence_citations=["https://www.fda.gov/.../warning-letters/"

"new-life-pharma-llc-725661-04142026"],

confidence=0.9, support=12, demo=True,

)
```

Worked example — analysis: *"FDA issued a warning letter to Jabil Inc., a large CDMO, citing aseptic processing violations."* (change_type: warning letter, product codes [LZG], submission types [510(k)]).

Scoring the three enforcement patterns:

pattern	triggers matched	coverage	× confidence	score	fits?
inspection-refusal-escalation	none	0/5	× 0.90	—	no
cdmo-warning-letter-cluster	cdmo, jabil	2/4 = 0.50	× 0.75	0.375	yes
glp1-compounding-crackdown	none	0/5	× 0.85	—	no

Result: the enforcement clone reasons from cdm0-warning-letter-cluster (relevance 0.375,

confidence 75%, fired on "cdmo", "jabil") and its prediction cites that pattern. This is the actual output of the algorithm, verified in a test run — deterministic, no LLM involved in retrieval.

10. Evidence-grounded prompting — pattern injection

Concept. The matched patterns are rendered into the prompt as the clone's evidence block, and the model is instructed to ground its prediction in them and name the ones it used. The result echoes `patterns_used`, so every prediction is traceable to evidence.

```
def respond_to_change(self, change_analysis):

    pattern_lines = [

        f"- {p.name} (relevance {score:.2f}, confidence {p.confidence:.0%}), "

        f"fired on: {' '.join(matched)}: {p.description} Evidence: {ev}"

        for p, score, matched in self.patterns

    ]

    question = (

        "A new FDA regulatory change has been analyzed (details in Context). "

        f"As a {self.profile['role']], predict how this stakeholder behaves in "

        "response: concrete actions, likely timelines, and what would change "

        "your prediction."

        "\n\nKnown behavior patterns for this stakeholder (distilled from FDA "

        "records). Ground your prediction in the ones that apply and name them:\n"

        + "\n".join(pattern_lines)
```

```

        + " End with a confidence level (high/medium/low) and why."

    )

    out = self.ask(question, context=json.dumps(change_analysis, indent=2)[:12000])

    return {

        "clone": self.profile["name"],

        "prediction": out["answer"],

        "citations": out["citations"],

        "patterns_used": [

            {"name": p.name, "score": score, "confidence": p.confidence,

             "matched_triggers": matched}

            for p, score, matched in self.patterns

        ],

    }

```

11. Fault-tolerant fan-out simulation

Concept. Every clone runs independently against the same analysis; one clone's failure is captured as an error payload instead of aborting the batch. Partial results beat no results.

```

def run_simulation(client, change_analysis, clones=None):

    clones = clones if clones is not None else build_clones(client)

    predictions = {}

    for clone in clones:

```

```

name = clone.profile["name"]

try:

    predictions[name] = clone.respond_to_change(change_analysis)

except Exception as exc: # one clone failing never kills the batch

    predictions[name] = {"clone": name, "error": str(exc)}

return {"change_summary": change_analysis.get("summary"),

        "predictions": predictions}

```

12. JSON Schema as LLM output contracts

Concept. The Task API accepts an output schema; the model must conform, which turns free-text generation into typed function output. CHANGE_ANALYSIS_SCHEMA defines the analysis record; BEHAVIOR_PATTERN_SCHEMA defines mined patterns.

```

CHANGE_ANALYSIS_SCHEMA = {

    "type": "object",

    "required": ["change_type", "summary", "severity", "confidence"],

    "properties": {

        "change_type": {"type": "string",

                        "enum": ["new_requirement", "tightened", "relaxed",

                                "clarification", "enforcement_shift"]},

        "summary": {"type": "string"},

        "affected_product_codes": {"type": "array", "items": {"type": "string"}},

```

```
"affected_submission_types": {"type": "array", "items": {"type": "string"}},

"severity": {"type": "integer", "minimum": 1, "maximum": 5},

"citations": {"type": "array", "items": {"type": "string"}},

"confidence": {"type": "number", "minimum": 0, "maximum": 1},

},

}
```

13. Graceful degradation — demo-mode architecture

Concept. The system must be fully demonstrable with no API key. On startup, if no key is configured and the store is empty, it seeds 8 demo events built from real FDA publications (real titles/URLs/dates; synthetic analyses labeled demo: true). The UI renders DEMO DATA callouts, and "Run analysis/simulation" flows work end to end against canned payloads.

```
@app.on_event("startup")

def _startup():

    if not settings.parallel_api_key and not store.list():

        seed_demo(settings.data_dir) # real publications, synthetic analyses
```

14. Thin API client (stdlib only)

Concept. One urllib-based client wraps every Parallel endpoint (Search, Extract, Task, Chat, FindAll, Monitor, Task Groups) behind typed methods. No SDK dependency, no heavy frameworks. Uncertain API fields are isolated here with defensive parsing and TODO markers, so verifying against real docs touches one file.

15. Deployment: FastAPI + Railway + GitHub auto-deploy

Concept. Standard 12-factor web service: Procfile runs uvicorn, config from environment (PARALLEL_API_KEY, DATA_DIR, WEBHOOK_BASE_URL), persistent volume for the JSON event store, Railway redeploys on every push to main.

Architecture recap

Sense (Search/Extract/Monitor) → events

→ Understand (Task + CHANGE_ANALYSIS_SCHEMA) → analyses

→ Clone (FindAll + Task → pattern library; Chat + matcher → predictions)

→ Simulate (fan-out, fault-tolerant)

→ Surface (FastAPI dashboard: overview, events, clones, monitors)

The key separation: **mining** (batch, expensive, LLM) vs. **matching** (deterministic, free, explainable) vs. **reasoning** (LLM, grounded in matched evidence). That is what makes the clones more than personas in a prompt.